

Documentație TEMA 1 APD 2025

Gherghel Ștefan-Ciprian 333CA

1. Feedback

Dificultatea temei nu a fost una ridicată și implementarea în sine nu a durat foarte mult timp dar a fost îngreunată de testarea pe sistemul meu slab (folosind WSL) care îmi oferea rezultate foarte variabile pe aceleași date de intrare (vorbind de timpii de rulare).

Mi-a plăcut faptul că rezolvarea finală nu a necesitat atât de multe elemente de sincronizare față așteptările mele inițiale, fiind necesară o minimă organizare a task-urilor în câteva etape.

Sigur puteam să obțin rezultate mai bune din punct de vedere al scalabilității dacă aveam mai mult timp și chef să schimb structuri de date sau să fac anumite sortări de mână dar acestea au fost îngreunate de faptul că trebuia să trimit pe Moodle de fiecare dată să verific rezultatele (pe local mă ajuta doar verificarea corectitudinii practic).

2. Strategia de paralelizare

Pentru paralelizarea implementării am folosit un **FixedThreadPool** în care am introdus task-uri pe care thread-urile să le execute. Am realizat **împărțirea pe etape** prin folosirea unor **CountDownLatch-uri** și folosind **design-ul general** pentru funcții / task-uri:

```
waitLatch.await()

// work

downLatch.countDown()
```

Prin acest design mă asigur că bucata de cod se execută după încheierea unei etape din trecut de care aceasta depinde și notifică terminarea execuției în cazul în care alte task-uri așteaptă de asemenea.

Am folosit **structuri de date concurente**:

- **BlockingQueue** - pentru distribuirea fișierelor și a articolelor între thread-uri
- **ConcurrentSkipListSet** – set folosit pentru a avea structurile ordonate în timp real și la final să fac doar printările
- **ConcurrentHashMap** – mapare concurentă folosită pentru numărarea duplicatelor și determinarea statisticilor privind autorii, limba și grupul
- **AtomicInteger** – pentru incrementări atomice
- **AtomicLong** – numărarea duplicatelor

Pentru structura de date ce stochează frecvențele fiecărui cuvânt nu am ales o variantă care să fie sortată în timp real (față de restul statisticilor) deoarece cu aceea variantă timpii de rulare erau mai slabi.

Utilizarea structurilor de mai sus este corectă și eficientă deoarece:

- **toate sunt thread-safe** fără a necesita blocări explicite (synchronized)
- **se împarte munca egal între thread-uri** în procesarea inițială prin folosirea cozilor blocante
- structurile sortate rezolvă **ordonarea cerută de temă** fără sortări adiționale – în cadrul implementării mi s-a părut dificil să distribui egal task-uri de sortare spre thread-uri și am ales această variantă
- **latch-urile** folosite **demarchează clar etapele** și nu permit rezultate diferite în cadrul rulărilor multiple

Etapele sunt următoarele:

I. Identificarea duplicatelor

În prima etapă, thread-urile (executând codul din ***MyThread.run()***) extrag **în paralel** căi de fișier din fileQueue, citesc fișierele JSON și extrag articolele în clasa corespunzătoare.

Pentru fiecare articol:

- **incrementează contorul apariției identificatorului** - uuidCount
- **incrementează contorul apariției titlului** - titleCount
- **stochează articolul în coada globală** - allArticles

La finalizarea fiecărui thread, CountdownLatch-ul corespunzător este decrementat. Această etapă pregătește datele necesare identificării articolelor duplicate.

II. Procesarea inițială a articolelor

După **finalizarea etapei 1** articolele sunt procesate **în paralel** prin următoarele acțiuni (ProcessArticles.run()):

- **incrementează contorul global pentru autorul articolului** - authorCount
- **actualizează structurile ce țin de categorii și limbă** – langGroup și categGroup
- procesarea textului și actualizarea structurii ce stochează **frecvența globală a cuvintelor** în cazul articolelor în limba engleză – wordFreq
- **adăugarea articolelor** în setul ordonat al articolelor unice - parsedUniqueArticles
- **actualizarea contorului de duplicate** dacă este cazul

III. Determinarea și printarea statisticilor generale

După **finalizarea etapei 2** se execută următoarele task-uri (**în paralel**) care se aplică **articolelor unice**:

- printarea **tuturor articolelor ordonate** - "all_articles.txt"
- printarea **fișierelor** ce grupează articolele **în funcție de limbă** - "language.txt"
- printarea **fișierelor** ce grupează articolele **în funcție de categorie** - "category.txt"
- determinarea **autorului cu cele mai multe articole (*)**
- determinarea **categoriei cu cele mai multe articole (*)**
- determinarea **limbii cu cele mai multe articole (*)**
- determinarea **cuvântului cu cele mai multe apariții** și **printarea ordonată cuvintelor cu frecvența lor (*)**

IV. Printarea statisticilor finale – "reports.txt"

Printarea statisticilor **după finalizarea** determinărilor **marcate cu (*)** din etapa 3.

3. Analiză de performanță și scalabilitate

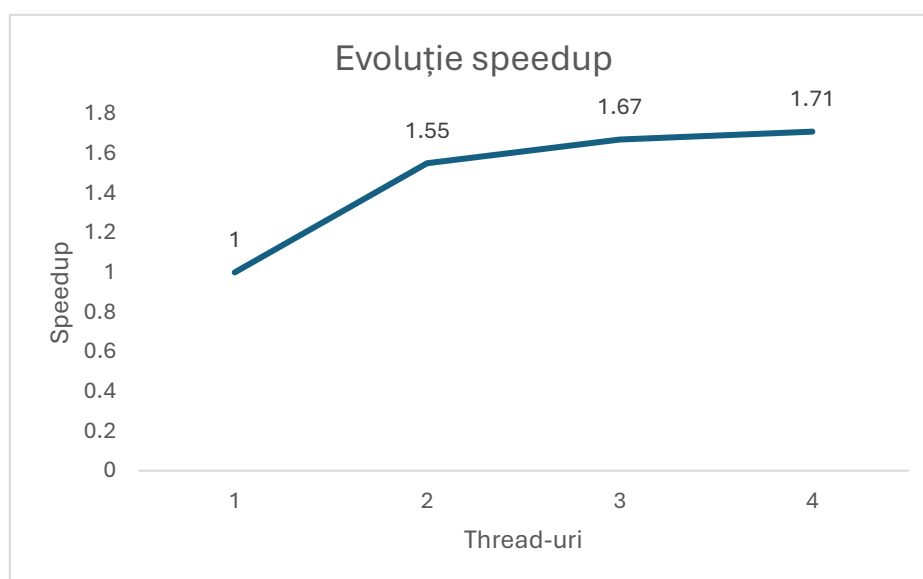
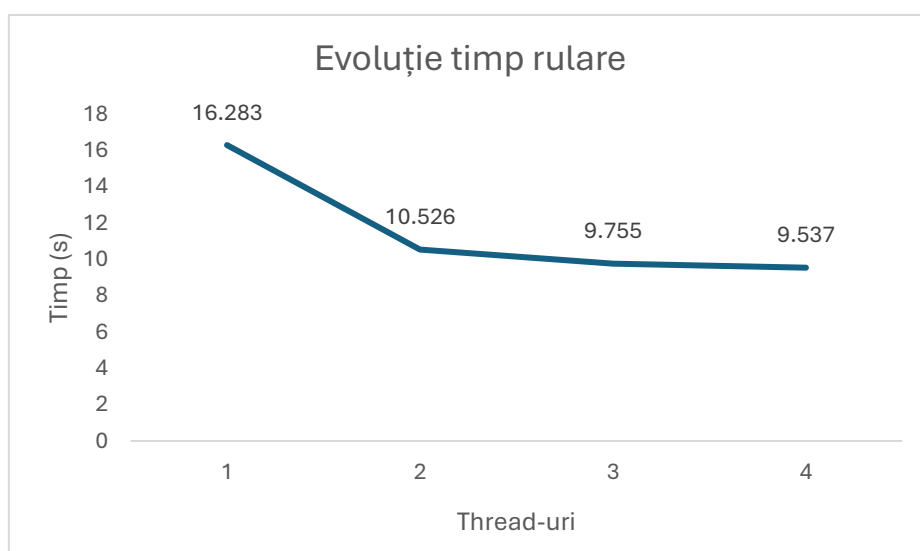
- Setup de testare:
 - Configurația sistemului gazdă:
 - Windows 10 Pro
 - 8 GB RAM
 - i5-7300HQ 2.50GHz 4 nuclee
 - Mediu de rulare:
 - WSL2 – Ubuntu 22.04
 - Java 17 (OpenJDK 17.0.15, 64-bit, Ubuntu 22.04)
 - Dimensiunea setului de date (specificațiile testului 5):
 - 13 789 fișiere cu articole care conțin 2-10 articole fiecare
 - 24 de categorii de interes
 - 140 de cuvinte de legătură
 - 19 limbi de interes

- Rezultate:

Am rulat de 10 ori fiecare configurație de test și am obținut următoarele rezultate:

Thread-uri	Timp 1 (s)	Timp 2 (s)	Timp 3 (s)	Timp 4 (s)	Timp 5 (s)	Timp 6 (s)	Timp 7 (s)	Timp 8 (s)	Timp 9 (s)	Timp 10 (s)	Timp mediu (s)
1	16.343	16.648	16.537	16.333	16.346	15.836	16.216	16.272	16.429	15.870	16.283
2	10.102	10.857	10.751	10.282	9.935	11.014	10.403	10.212	11.402	10.304	10.526
3	9.174	9.886	9.777	9.600	10.481	10.224	9.498	10.679	9.310	8.918	9.755
4	8.967	9.976	9.462	9.646	9.665	9.065	10.106	9.216	9.721	9.542	9.537

Thread-uri	Timp mediu (s)	Speedup	Eficiență
1	16.283	1	1
2	10.526	1.55	0.78
3	9.755	1.67	0.56
4	9.537	1.71	0.43



- Analiză și concluzii:
 - **cea mai mare creștere a performanței** are loc la trecerea de la **1** thread la **2** thread-uri (**$S = 1.55$, $E = 0.78$**)
 - pentru **3** și **4** thread-uri S crește mai lent și eficiența scade deoarece **crește competiția pe structurile de date**
 - la **4** thread-uri (**limita sistemului**) avem în vedere limitarea că programul nu rulează singur pe procesor
 - **număr optim de thread-uri** (cel mai bun timp) – **4**
 - posibile limitări:
 - **competiție ridicată pe structurile globale**, în special pe cele ordonate
 - **etapa 1 este foarte dependentă de I/O**, limitând paralelizarea
 - **un overhead de acces la disc al WSL**
 - **dimensiunea setului de date** (s-ar putea să obținem rezultate mai bune pe un set de date mai mare)
 - **inegalitatea task-urilor din etapa 3** – ele se execută în paralel dar nu reprezintă un volum de muncă tocmai egal
 - **partea neparalelizată** – citirea fișierelor de intrare și popularea cozii de căi